

Middleware in Express.js – Request Logging

Student Name: P. Dharanii**Experiment No.:** 6-8**Course:** Full Stack Web Development**Course Code:** 23CM4121

① Title

Middleware in Express.js — Request Logging

② Aim

To understand middleware in Express.js and develop a custom middleware that logs incoming HTTP requests.

③ Objective

- To understand the concept of middleware in Express.js.
- To understand the request-response cycle.
- To create and register custom middleware.
- To log the HTTP method and requested URL of incoming requests.
- To use `next()` to pass control to the route handler.
- To verify the middleware using a browser and the terminal.

④ Theory

Middleware in Express.js is a function that has access to the request object, the response object and the `next()` function. It can run code, modify the request or response, end the request-response cycle, or pass control to the next middleware or route.

Middleware is commonly registered with `app.use()`. In this experiment a custom logging middleware records the time, the HTTP method and the requested URL of every incoming request.

Stage	Request-response cycle
1	Client / browser sends a request
2	Express server receives it
3	Logging middleware runs and prints the log line
4	<code>next()</code> passes control onward
5	Route handler creates the response
6	Response is sent back to the browser

⑤ Algorithm / Procedure

1. Create a new Node.js project folder.
2. Initialize the project using `npm init -y`.
3. Install Express using `npm install express`.
4. Import the Express module into `app.js`.
5. Create an Express application.
6. Define a custom `requestLogger` middleware function.
7. Log the current time, HTTP method and requested URL.
8. Call `next()` so that the request continues to the route.
9. Create a GET route for the root URL.
10. Start the Express server on port 3000.
11. Open `http://localhost:3000/` in a browser.
12. Observe the webpage response and the request log in the terminal.
13. Refresh the page and verify that another GET request is logged.

6 Program

Project structure

```
Q8-Express-Middleware/  
├─ app.js  
├─ package.json  
├─ package-lock.json  
└─ node_modules/      (generated by NPM – not uploaded to GitHub)
```

app.js

```
// Experiment 6-8: custom request-logging middleware in Express.js  
const express = require("express");  
  
const app = express();  
const PORT = 3000;  
let requestCount = 0;  
  
// Custom middleware: runs for every incoming request  
function requestLogger(req, res, next) {  
  requestCount++;  
  const time = new Date().toLocaleTimeString();  
  console.log("[ " + time + " ] Request #" + requestCount + " -> " + req.method + " " + req.url);  
  next(); // hand over to the next middleware / route  
}  
  
// Register the middleware before the routes  
app.use(requestLogger);  
  
// Route for the root URL  
app.get("/", (req, res) => {  
  res.send(  
    "<h1>Middleware Demo</h1>" +  
    "<p>Every request to this server is logged in the terminal.</p>"  
  );  
});  
  
// A second route to show that different URLs are logged  
app.get("/about", (req, res) => {  
  res.send("<h1>About Page</h1><p>This page is also logged by the middleware.</p>");  
});  
  
// Start the server  
app.listen(PORT, () => {  
  console.log("Server is listening on http://localhost:" + PORT);  
});
```

7 Code Explanation

Code / Method	Explanation
<code>const express = require("express")</code>	Imports the Express.js package.
<code>const app = express()</code>	Creates the Express application object.
<code>let requestCount = 0</code>	Counter that stores how many requests the server has received.
<code>function requestLogger(req, res, next)</code>	The custom middleware. It receives the request, the response and the <code>next</code> function.
<code>new Date().toLocaleTimeString()</code>	Gets the current local time so that each log line shows when the request arrived.
<code>req.method, req.url</code>	The HTTP method (GET, POST ...) and the requested URL; both are printed in the log together with the request number.
<code>next()</code>	Passes control to the next middleware or route handler. Without it the request would stay waiting and the browser would not get a response.
<code>app.use(requestLogger)</code>	Registers the middleware for every request. It is placed before the routes so that it runs first.
<code>app.get("/", ...) and app.get("/about", ...)</code>	Route handlers for the root URL and the about page; each sends an HTML response with <code>res.send()</code> .
<code>app.listen(PORT, ...)</code>	Starts the server on port 3000 and prints a start-up message.

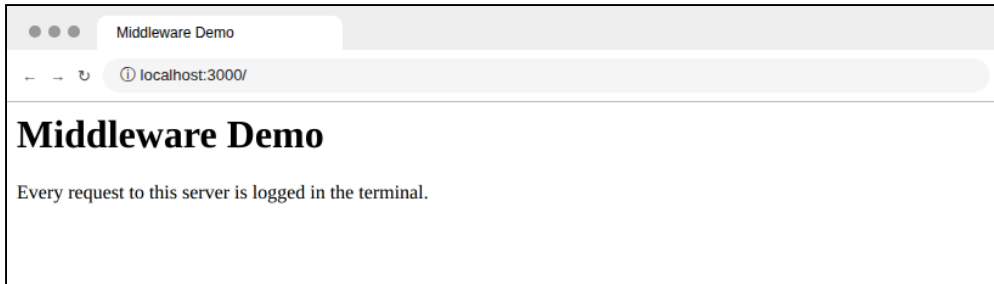
8 Test Cases and Execution Data

The application was run and tested using a browser and the terminal. Time values in the log depend on when each request is made.

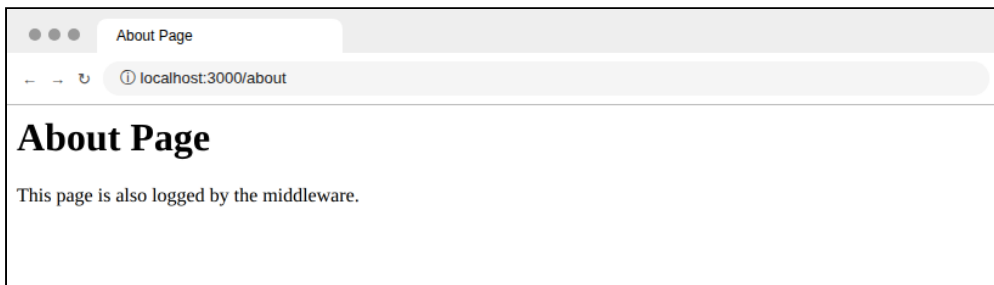
Test Case	Test / Input	Expected Result	Actual Result	Status
TC-01	Run npm install express	Express is installed without errors and listed in package.json dependencies.	Installation completed; "express" listed under dependencies.	Pass
TC-02	Run node app.js	Server starts on port 3000 and prints the start-up message.	Server is listening on <code>http://localhost:3000</code> .	Pass
TC-03	Open http://localhost:3000/ in the browser	Page "Middleware Demo" is displayed and the log shows <code>GET /</code> .	Page displayed correctly; log: <code>[4:47:52 PM] Request #1 -> GET /</code>	Pass
TC-04	Browser requests the favicon (automatic)	Middleware also logs <code>GET /favicon.ico</code> .	Log: <code>[4:47:52 PM] Request #2 -> GET /favicon.ico</code>	Pass
TC-05	Refresh the root page	A new <code>GET /</code> line appears with the next request number.	Log: <code>[4:47:53 PM] Request #3 -> GET /</code>	Pass
TC-06	Open /about	"About Page" is displayed and <code>GET /about</code> is logged.	Page displayed; log: <code>[4:47:54 PM] Request #4 -> GET /about</code>	Pass
TC-07	Open an undefined URL /contact (special case)	Middleware still logs the request, then Express answers 404 "Cannot GET /contact".	404 page displayed; log: <code>[4:47:55 PM] Request #5 -> GET /contact</code>	Pass
TC-08	Send a POST request to / (special case)	Request is logged with method POST; no POST route exists, so the response is 404.	Response status 404; log: <code>[4:47:55 PM] Request #6 -> POST /</code>	Pass

9 Output

Output 1: Root page (/) in the browser



Output 2: About page (/about) in the browser



Output 3: Server terminal showing the request log

```
Terminal - server console

student@vm:~/Q8-Express-Middleware/src$ node app.js
Server is listening on http://localhost:3000
[4:47:52 PM] Request #1 -> GET /
[4:47:52 PM] Request #2 -> GET /favicon.ico
[4:47:53 PM] Request #3 -> GET /
[4:47:54 PM] Request #4 -> GET /about
[4:47:55 PM] Request #5 -> GET /contact
[4:47:55 PM] Request #6 -> POST /
```

Observed execution results

Item	Observed Value
Server	Server is listening on http://localhost:3000
Browser output (/)	Middleware Demo — Every request to this server is logged in the terminal.
Logged request 1	GET / at 4:47:52 PM
Logged request 2	GET /favicon.ico at 4:47:52 PM
Logged request 3	GET / at 4:47:53 PM (page refresh)
Logged request 4	GET /about at 4:47:54 PM
Logged request 5	GET /contact at 4:47:55 PM (404 response)
Logged request 6	POST / at 4:47:55 PM (404 response)

10 Result

Middleware in Express.js was successfully demonstrated by developing a custom request-logging middleware. The middleware logged every incoming HTTP request, used `next()` to continue the request-response cycle, and the application was verified through the browser and the terminal.

End of Observation Record